In God we trust

عنوان پروژه : *مدیریت حافظه در xv6*

دانشجویان : *ریحانه عین‌الهی - بهاره عین‌الهی - مهدیس میرزایی*

شماره پروژه : *پروژه پنجم(امتیازی) آزمایشگاه سیستم عامل*

لینک گیت‌هاب : گیت‌هاب پروژه پنجم آزمایشگاه

دانشکده مهندسی برق و کامپیوتر - دانشگاه تهران

دی ۱۴۰۴

بخش اول: تحلیل هسته و قفل‌های پیشرفته

سوالات تئوری

پرسش 1 : 1) راجع به مفهوم ناحیه مجازی در لینوکس به طور مختصر توضیح داده و آن را با xv6 مقایسه کنید. صفحه های محافظ به چه منظور استفاده میشوند؟

پاسخ :

Virtual Memory Area (VMA) در Linux

در Linux، یک **Virtual Memory Area (VMA)** نشان‌دهندهٔ یک بازهٔ پیوسته از آدرس‌های مجازی در فضای آدرس یک process است. این ناحیه توسط ساختار `vm_area_struct` تعریف می‌شود.

- **Characteristics:** هر VMA دارای آدرس شروع و پایان، permissionهای مشخص (read, write, execute) و یک هدف معین است (مثلاً stack، heap یا file-backed mapping).
- **Abstraction**: لینوکس از VMAها برای track کردن این‌که *چه چیزی باید* در memory باشد استفاده می‌کند. اگر یک process به آدرسی دسترسی پیدا کند، kernel لیست VMAها را بررسی می‌کند. اگر آدرس داخل یک VMA باشد ولی هنوز در page table وجود نداشته باشد، یک **page fault** رخ می‌دهد تا داده به‌صورت lazy load شود.
- **Management:** چندین VMA با استفاده از یک **red-black tree** (برای lookup سریع) و یک **linked list** (برای scan خطی) مدیریت می‌شوند.

در xv6 هیچ ساختار high-levelای به نام VMA وجود ندارد. Kernel به‌سادگی فرض می‌کند process مالک تمام آدرس‌ها 0 از تا sz است. اگر به آدرسی خارج ازsz دسترسی داده شود، error رخ می‌دهد.

مقایسه Linux در مقابل xv6

Feature	Linux VMA-based	xv6
Mapping Unit	از `vm_area_struct` برای نمایش segment ها استفاده می‌کند.	از یک متغیر ساده به نام (size `sz`) برای track کردن انتهای user memory استفاده می‌کند.
Allocation	**(Lazy (On-demand.** حافظه فقط زمانی allocate می‌شود که page fault رخ دهد.	**Eager.** حافظه معمولاً بلافاصله در `sbrk()` یا `exec()` allocate می‌شود.
Structure	پیشرفته (red-black tree و linked list).	حداقلی (یک address space خطی از 0 تا `sz`).
Flexibility	پشتیبانی از hole در address space و file-backed mapping.	address space معمولاً یک block پیوسته از pageهای allocateشده است.

هدف Guard Pageها

یک **Guard Page** یک memory page است که به‌صورت **no-access** mark می‌شود و بین نواحی مهم memory قرار می‌گیرد.

- **Stack Overflow Detection:** رایج‌ترین استفاده در انتهای stack است. اگر stack بیش از حد grow کند و به guard page برسد، hardware یک page fault یا segmentation fault ایجاد می‌کند. این کار جلوی overwrite شدن silent هیپ یا سایر dataها را می‌گیرد.
- **Buffer Overflow Prevention:** با قرار دادن guard page بین bufferها یا VMAهای مختلف، OS تضمین می‌کند که یک out-of-bounds write در یک ناحیه، ناحیهٔ مجاور را corrupt نکند.
- **Security:** Guard pageها مثل یک **tripwire** برای malicious code یا bugهایی عمل می‌کنند که سعی دارند memory را به‌صورت sequential scan یا overflow کنند.

پرسش 2 : چرا ساختار سلسله مراتبی منجر به کاهش مصرف حافظه می گردد؟

پاسخ :

در ساختار سلسله‌مراتبی، processها و taskها می‌توانند با اشتراک‌گذاری کدها و داده‌ها، مصرف حافظه را کاهش دهند. سیستم‌عامل با استفاده از mapping مناسب، بخش‌های مورد نیاز هر فرآیند را به صفحات فیزیکی حافظه مرتبط می‌کند و از مصرف اضافی حافظه جلوگیری می‌کند.

این ساختار به سیستم‌عامل اجازه می‌دهد تا صفحات مختلف حافظه به ترتیب دسترسی قرار گیرند و به‌صورت دینامیک مدیریت شوند. به عبارت دیگر، فقط آن بخش‌هایی از حافظه که مورد نیاز فرآیند هستند، بارگذاری می‌شوند و دسترسی به داده‌ها سریع‌تر و بهینه‌تر صورت می‌گیرد.

پرسش 3: مفهوم هر بیت یک مدخل (۳۲ بیتی) در هر سطح چیست؟ چه تفاوتی میان آنها وجود دارد؟

پاسخ :

در هر دو سطح (Page Directory و Page Table)، ۲۰ بیت اول برای ذخیره آدرس فیزیکی استفاده می‌شوند؛ با این تفاوت که در سطح اول به آدرس شروع جدول صفحه و در سطح دوم به آدرس فیزیکی قاب (Frame) اشاره دارند. ۱۲ بیت دیگر برای تنظیمات دسترسی هستند:

- **P Present:** موجود بودن صفحه در حافظه فیزیکی.
- **W Writable:** اجازه نوشتن در صفحه.
- **U User:** اجازه دسترسی پردازه‌های سطح کاربر.
- **تفاوت کلیدی:** تنها تفاوت میان مدخل‌های این دو سطح در بیت **D Dirty** است؛ این بیت در سطح جدول صفحه (Table) کاربردی ندارد اما در دایرکتوری صفحه (Directory) مشخص می‌کند که آیا صفحه باید برای اعمال تغییرات در دیسک نوشته شود یا خیر.

پرسش 4 : تابع kalloc چه نوع حافظه‌ای تخصیص میدهد؟ (فیزیکی یا مجازی)

پاسخ :

تابع `kalloc` وظیفه تخصیص **حافظه فیزیکی** را بر عهده دارد. این تابع یک صفحه ۴۰۹۶ بایتی از حافظه فیزیکی آزاد را جدا کرده و اشاره‌گری به آن برمی‌گرداند که هسته می‌تواند از آن استفاده کند

پرسش 5 : تابع mappages چه کاربردی دارد؟

پاسخ :
تابع mappages قلب مدیریت حافظه مجازی در xv6 است. وظیفه آن «ثبتِ نگاشت» است.
وقتی سیستم‌عامل می‌خواهد به یک پردازه بگوید «آدرس مجازی V باید به آدرس فیزیکی P اشاره کند»، از این تابع استفاده می‌کند.
ورودی‌ها: اشاره‌گر به جدول صفحه، آدرس مجازی شروع، طول ناحیه، آدرس فیزیکی متناظر و مجوزها (Permissions).
این تابع آدرس مجازی را صفحه به صفحه جلو می‌برد، برای هر صفحه تابع walkpgdir را صدا می‌زند تا ورودی (PTE) مربوطه را پیدا کند و سپس آدرس فیزیکی و پرچم‌ها را در آن PTE می‌نویسد.

پرسش 6: آیا تمام قسمت های حافظه مجازی از آدرس صفر تا proc->sz از طریق کاربر قابل استفاده اند؟

پاسخ :
خیر، تمام این فضا قابل استفاده (Accessible) نیست. در xv6، درست زیر پشته (Stack) پردازه، یک صفحه به نام Guard Page قرار می‌گیرد.

- این صفحه در جدول صفحه علامت‌گذاری شده اما فاقد مجوز دسترسی کاربر (PTE_U ندارد یا Present نیست).
- علت: اگر پشته پردازه بیش از حد رشد کند (مثلاً در یک تابع بازگشتی بی‌نهایت)، به جای اینکه پشته روی بخش «داده‌های برنامه» (Data Segment) بازنویسی شود و باعث رفتار غیرقابل پیش‌بینی شود، پردازه با این صفحه محافظ برخورد کرده و سیستم‌عامل آن را متوقف (Terminated) می‌کند.

پرسش 7 : راجع به تابع walkpgdir توضیح دهید. این تابع چه عمل سخت افزاری را شبیه سازی میکند؟

پاسخ :

وظیفه اصلی این تابع، یافتن آدرس مدخل جدول صفحه (PTE) برای یک آدرس مجازی داده شده است. این تابع در واقع همان فرآیند «پیمایش جدول صفحه» را که در سخت‌افزار توسط MMU (واحد مدیریت حافظه) انجام می‌شود، به صورت نرم‌افزاری پیاده‌سازی می‌کند.

پرسش 8: توابع allocuvm و mappages که در ارتباط با حافظهی مجازی هستند را توضیح دهید.

پاسخ :

تابع mappages یک حافظه مجازی را به حافظه ی فیزیکی متصل میکند.
تابع allocuvm ناحیه حافظه مجازی یک پردازه را از oldsz به newsz افزایش می دهد. این تابع تا زمانی که اندازه ی حافظه ی اولیه به مقدار خواسته شده برسد، حافظه فیزیکی allocate میکند و آن را با استفاده از تابع mappages به یک حافظه مجازی در پردازه متصل میکند.

پرسش 9: شیوه‌ی بارگذاري برنامه در حافظه توسط فراخوانی سیستمی exec را شرح دهید.

پاسخ :

مراحل بارگذاری یک برنامه توسط `exec` به شرح زیر است:

۱. باز کردن فایل و بررسی قالب ELF

ابتدا فایل اجرایی از طریق مسیر (path) داده شده پیدا شده و هدر آن خوانده می‌شود. xv6 بررسی می‌کند که آیا این فایل دارای قالب **ELF** هست یا خیر. اگر فایل معتبر نباشد، عملیات متوقف می‌شود.

۲. ایجاد جدول صفحه جدید (Memory Isolation)

به جای تغییر مستقیم جدول صفحه فعلی، `exec` ابتدا یک جدول صفحه (Page Table) جدید و خالی با استفاده از تابع `setupkvm` ایجاد می‌کند. این کار باعث می‌شود که اگر در میانه راه بارگذاری برنامه خطایی رخ داد، فرآیند قبلی همچنان سالم باقی بماند و سیستم دچار فروپاشی نشود.

۳. تخصیص حافظه و بارگذاری بخش‌های برنامه (Segments)

فایل‌های ELF دارای بخش‌های مختلفی (مانند `.text` برای کد و `.data` برای داده‌های مقداردهی شده) هستند. برای هر بخش (Segment) که نیاز به بارگذاری دارد، تابع `allocuvm` فراخوانی می‌شود تا حافظه فیزیکی مورد نیاز تخصیص یابد و نگاشت‌های آن در جدول صفحه جدید ایجاد شود. سپس با استفاده از تابع `loaduvm` محتویات هر بخش از دیسک به آدرس‌های حافظه مجازی مشخص شده کپی می‌شود.

۴. ایجاد صفحه محافظ (Guard Page) و پشته کاربر (User Stack)

بعد از بارگذاری کد و داده، `exec` فضای پشته (Stack) را تنظیم می‌کند:

تخصیص دو صفحه: سیستم‌عامل دو صفحه حافظه تخصیص می‌دهد.

صفحه محافظ (Guard Page): صفحه پایینی به عنوان "Guard Page" در نظر گرفته می‌شود. با استفاده از تابع `clearpteu`، دسترسی سطح کاربر از این صفحه گرفته می‌شود. این کار برای امنیت است؛ اگر پشته بیش از حد رشد کند و به این صفحه برسد، سخت‌افزار خطا (Fault) داده و از بازنویسی بخش داده‌ها جلوگیری می‌کند. صفحه بالایی به عنوان پشته واقعی در اختیار کاربر قرار می‌گیرد.

۵. آماده‌سازی آرگومان‌ها (Argument String Copy)

رشته‌های ورودی برنامه (`argv`) در بالای پشته کپی می‌شوند. همچنین اشاره‌گرهایی به این رشته‌ها و تعداد آن‌ها (`argc`) در پشته قرار می‌گیرند تا برنامه جدید هنگام شروع اجرا به آن‌ها دسترسی داشته باشد.

۶. نهایی‌سازی و تغییر جدول صفحه

جدول صفحه قدیمی فرآیند با استفاده از توابعی مانند `freevm` آزاد می‌شود. جدول صفحه جدید به عنوان جدول اصلی فرآیند (`proc->pgdir`) جایگزین می‌گردد. مقدار **EIP** در `trapframe` به آدرس نقطه شروع برنامه (Entry Point) که در هدر ELF آمده است، تغییر می‌یابد.اندازه فرآیند (`proc->sz`) به‌روزرسانی می‌شود.

پرسش 10: در مورد الگوریتم های مورد استفاده در سیستم عامل هاي ویندوز ولینوکس تحقیق کنید و به طور خلاصه کاربرد های هر کدام را ذکر کنید.

پاسخ :

لینوکس: از الگوریتم Clock (نسخه بهبود یافته LRU) با استفاده از دو لیست Active و Inactive استفاده می‌کند تا صفحاتی که کمتر مورد استفاده قرار گرفته‌اند را شناسایی و جایگزین کند.

ویندوز: از مفهوم Working Set استفاده می‌کند. در این سیستم، برای هر پردازه یک حداقل و حداکثر تعداد صفحه در نظر گرفته می‌شود و با استفاده از الگوریتم‌هایی مانند Clock یا FIFO در بازه‌های زمانی خاص، صفحاتی که نرخ دسترسی پایینی دارند از حافظه خارج می‌شوند.

سوالات عملی (سوال یازدهم و خروجی ها)

برنامه اول:

```
0: 0FIFO,
1: LRU,
2: LFU,
3: Clock
```

نتیجه: نسبت Hit برای همه‌ی الگوریتم‌ها ۰٪ است.

تحلیل: این نتیجه قابل انتظار است. چون به‌صورت sequential به ۱۱ صفحه‌ی unique دسترسی پیدا می شود. از آن‌جا که table فقط ظرفیت نگه‌داری ۴ صفحه را دارد، زمانی که به صفحه‌ی ۵ می‌رسد، صفحه‌ی ۰ قبلاً evicted شده است. چون هیچ صفحه‌ای دوباره revisit نمی‌شود، وقوع یک «Hit» غیرممکن است.

```
$ test1 0
--- Program 1: Testing Algorithm 0 ---
Accessed Page 0 (Virtual Addr: 3000)
Accessed Page 1 (Virtual Addr: 4000)
Accessed Page 2 (Virtual Addr: 5000)
Accessed Page 3 (Virtual Addr: 6000)
Accessed Page 4 (Virtual Addr: 7000)
Accessed Page 5 (Virtual Addr: 8000)
Accessed Page 6 (Virtual Addr: 9000)
Accessed Page 7 (Virtual Addr: A000)
Accessed Page 8 (Virtual Addr: B000)
Accessed Page 9 (Virtual Addr: C000)
Accessed Page 10 (Virtual Addr: D000)
Hit Count: 0
total access: 11
Hit Ratio: 0%
Execution Time: 4 ticks

$ test1 1
--- Program 1: Testing Algorithm 1 ---
Hit Count: 0
total access: 11
Hit Ratio: 0%
Execution Time: 5 ticks

$ test1 2
--- Program 1: Testing Algorithm 2 ---
Hit Count: 0
total access: 11
Hit Ratio: 0%
Execution Time: 5 ticks

$ test1 3
--- Program 1: Testing Algorithm 3 ---
Hit Count: 0
total access: 11
Hit Ratio: 0%
Execution Time: 5 ticks
```

برنامه دوم:

0) FIFO : صفحه‌ی i، صفحه‌ی i+1 را درست قبل از زمانی که به آن نیاز می‌شود evict می‌کند.
1) LRU : در یک cycle کامل، همیشه «Least Recently Used» همان صفحه‌ی بعدی در sequence است.
2) LFU : در این الگوریتم ۶۰٪ — frequencyها به‌مرور جمع می‌شوند و باعث می‌شوند ۴ صفحه در cache باقی بمانند، در حالی که فقط ۱ صفحه مدام swap می‌شود.
3) Clock : در این الگوریتم ۰٪ — عملاً مثل FIFO رفتار می‌کند، چون هر صفحه در هر cycle فقط یک‌بار visit می‌شود و همه‌ی bitهای «second chance» پاک می‌شوند.
الگوریتم‌هایی که hit ratio آن‌ها ۰٪ است، حدود ۲۶ تا ۲۹ ticks زمان برده‌اند، چون تقریباً در هر access یک disk swap انجام داده‌اند.

```
$ test2 0
--- Program 2: Cyclic 5-Page Access using Algo 0 ---
Loop 0: Read Page 0 (val: 0)
Loop 0: Read Page 1 (val: 100)
Loop 0: Read Page 2 (val: 200)
Loop 0: Read Page 3 (val: 300)
Loop 0: Read Page 4 (val: 400)
Loop 1: Read Page 0 (val: 0)
Loop 1: Read Page 1 (val: 100)
Loop 1: Read Page 2 (val: 200)
Loop 1: Read Page 3 (val: 300)
Loop 1: Read Page 4 (val: 400)
Loop 2: Read Page 0 (val: 0)
Loop 2: Read Page 1 (val: 100)
Loop 2: Read Page 2 (val: 200)
Loop 2: Read Page 3 (val: 300)
Loop 2: Read Page 4 (val: 400)
Loop 3: Read Page 0 (val: 0)
Loop 3: Read Page 1 (val: 100)
Loop 3: Read Page 2 (val: 200)
Loop 3: Read Page 3 (val: 300)
Loop 3: Read Page 4 (val: 400)
Hit Count: 0
total access: 20
Hit Ratio: 0%
Execution Time: 28 ticks

$ test2 1
--- Program 2: Cyclic 5-Page Access using Algo 1 ---
Hit Count: 0
total access: 20
Hit Ratio: 0%
Execution Time: 26 ticks

$ test2 2
--- Program 2: Cyclic 5-Page Access using Algo 2 ---
Hit Count: 12
total access: 20
Hit Ratio: 60%
Execution Time: 18 ticks

$ test2 3
--- Program 2: Cyclic 5-Page Access using Algo 3 ---
Hit Count: 0
total access: 20
Hit Ratio: 0%
Execution Time: 29 ticks
```

برنامه سوم:

نتیجه: حدود ۹۷ تا ۹۸ درصد Hit Ratio برای همه‌ی الگوریتم‌ها.

تحلیل: این نتیجه کاملاً قابل انتظار است. وقتی مجموعه‌ی «Hot» از pageها به‌طور کامل داخل table چهاراسلاتی جا می‌گیرد، تفاوت بین الگوریتم‌ها تقریباً از بین می‌رود، چون بعد از load شدن، pageها دیگر نیازی به evict شدن ندارند.

```
$ test3 0
--- Program 3: Hot/Cold Access Patterns (Algo: 0) ---
Hit Count: 14848
total access: 15250
Hit Ratio: 97%
Execution Time: 151 ticks

$ test3 1
--- Program 3: Hot/Cold Access Patterns (Algo: 1) ---
Hit Count: 14871
total access: 15250
Hit Ratio: 97%
Execution Time: 141 ticks

$ test3 2
--- Program 3: Hot/Cold Access Patterns (Algo: 2) ---
Hit Count: 15000
total access: 15250
Hit Ratio: 98%
Execution Time: 108 ticks

$ test3 3
--- Program 3: Hot/Cold Access Patterns (Algo: 3) ---
Hit Count: 14850
total access: 15250
Hit Ratio: 97%
Execution Time: 144 ticks
```

برنامه چهارم:

FIFO (0)

۲۸٪ — بدترین حالت. ترتیب ورود این‌جا معیار خوبی نیست و pageهای مفید را فقط به‌خاطر قدیمی‌تر بودن evict می‌کند.

LRU (1)

۵۶٪ — خوب. در نگه‌داشتن زیرمجموعه‌ی «active sequence» در memory مؤثر است و خیلی بهتر از FIFO عمل می‌کند.

LFU (2)

۷۱٪ — بهترین. به‌نظر می‌رسد بعضی pageها بیشتر تکرار می‌شوند. LFU این pageهای «heavy» را به‌خاطر access count بالا نگه می‌دارد و swapها را کم می‌کند.

Clock (3)

۲۸٪ — ضعیف. مثل FIFO رفتار می‌کند. این نشان می‌دهد bitهای «second chance» قبل از reuse شدن پاک می‌شوند.

Latency Impact: به Execution Time توجه کنید؛ هر ۱٪ افزایش در Hit Ratio حدود ۸–۹ ticks صرفه‌جویی می‌کند. LFU تقریباً دو برابر سریع‌تر از FIFO است، چون تعداد زیادی disk I/O را حذف می‌کند.

Clock در برابر LRU: از نظر تئوری، Clock باید به ۵۶٪ (LRU) نزدیک باشد. این‌که نتیجه‌اش مثل ۲۸٪ (FIFO) شده، یعنی sequence آن‌قدر طولانی است که reference_bit قبل از revisit پاک می‌شود.

برتری LFU:

Hit Ratio برابر ۷۱٪ نشان می‌دهد توزیع accessها «Non-Uniform» است و تعداد کمی page بخش عمده‌ی ۷۰۰ access را تشکیل می‌دهند.

```
$ test4 0
--- Program 4: Specific Sequence Access (Algo: 0) ---
Hit Count: 200
total access: 700
Hit Ratio: 28%
Execution Time: 539 ticks
$ test4 1
--- Program 4: Specific Sequence Access (Algo: 1) ---
Hit Count: 398
total access: 700
Hit Ratio: 56%
Execution Time: 365 ticks
$ test4 2
--- Program 4: Specific Sequence Access (Algo: 2) ---
Hit Count: 500
total access: 700
Hit Ratio: 71%
Execution Time: 282 ticks
$ test4 3
--- Program 4: Specific Sequence Access (Algo: 3) ---
Hit Count: 200
total access: 700
Hit Ratio: 28%
Execution Time: 524 ticks
```